



INDIA.RUNS

Build what next India runs on

INTELLIGENT CANDIDATE DISCOVERY & RANKING

*Ranking 100,000 candidates the way a great recruiter would —
by reading career evidence, not counting keywords.*

TEAM

InnoCoders

TEAM LEADER

Devansh Wadhwani

PROBLEM STATEMENT

**Senior AI / ML Engineer
Candidate Ranking**

SOLUTION OVERVIEW

02

A recruiter-grade ranking engine

What is our proposed solution?

A deterministic, rule-based engine that scores every candidate across five recruiter dimensions, defends against planted profiles, and outputs an auditable top-100 — in 31 seconds, with zero dependencies.

WHAT WE BUILT

- Scores 100K profiles across five weighted, JD-derived dimensions.
- Pure Python standard library — no GPU, no network, no model downloads.
- Emits a validated top-100 CSV with per-candidate reasoning.

WHAT MAKES IT DIFFERENT

- Career descriptions are ground truth; skills are cross-validated against them — keyword-stuffers collapse.
- Built-in honeypot defense flags internally impossible profiles.
- Identical input yields byte-identical output — fully reproducible.

JD UNDERSTANDING

From job description to relevance

03

REQUIREMENTS EXTRACTED FROM THE JD

- Senior AI/ML engineer — retrieval, ranking & recommendation systems.
- 6–8+ yrs with production ML deployment and strong coding.
- Depth in NLP, search and vector retrieval.
- Location preference: Pune / Noida.

STATED DISQUALIFIERS

Non-technical roles · consulting-only careers · research without production · CV / speech specialists without NLP crossover · ghost candidates.

SIGNALS THAT MOST DETERMINE RELEVANCE

- 1 Career-description evidence**
the single hardest signal to fake
- 2 Role & title trajectory**
current title + career peak, recency-weighted
- 3 Production-deployment proof**
shipped systems, not just research
- 4 Company quality & tenure**
product companies, stability over hopping
- 5 Availability & responsiveness**
open-to-work, notice period, response rate

RANKING METHODOLOGY

Retrieve, score, rank

04

FIVE WEIGHTED COMPONENTS



Skills are self-reported, so each is cross-validated against career text before it can lift a score.

FINAL SCORE

$$\text{base} \times (0.70 + 0.30 \cdot \text{location}) \times \text{penalty} \times \text{honeypot flag} \times 0.02$$

Heuristics, not a black box

Word-boundary keyword matching, a four-tier title taxonomy, an experience curve peaking at 6–8 years.

Penalties stack multiplicatively

Each JD-stated disqualifier compounds, so weak profiles fall fast rather than averaging out.

Deterministic tie-break

Equal scores are ordered by candidate_id ascending — exactly as the spec requires.

EXPLAINABILITY & DATA VALIDATION

05

Trustworthy, and impossible to game

EXPLAINABILITY

- Every row carries a 1–2 sentence justification built only from facts in that profile.
- Honest concerns surfaced: long notice, low response rate, off-preference location.
- Phrasing varies by rank band — never templated, never repetitive.
- Zero hallucinated skills: each string is regex-verified against the profile.

HONEYPOT DEFENSE

Every profile is verified against itself.

- **Date arithmetic**

A role's claimed duration contradicts its own start/end dates.

- **Expert, zero use**

3+ skills at expert proficiency with 0 months of usage.

- **Experience span**

Stated years exceed the entire career history by 2+ years.

- **Tech anachronism**

Skill durations predate the technology's public release.

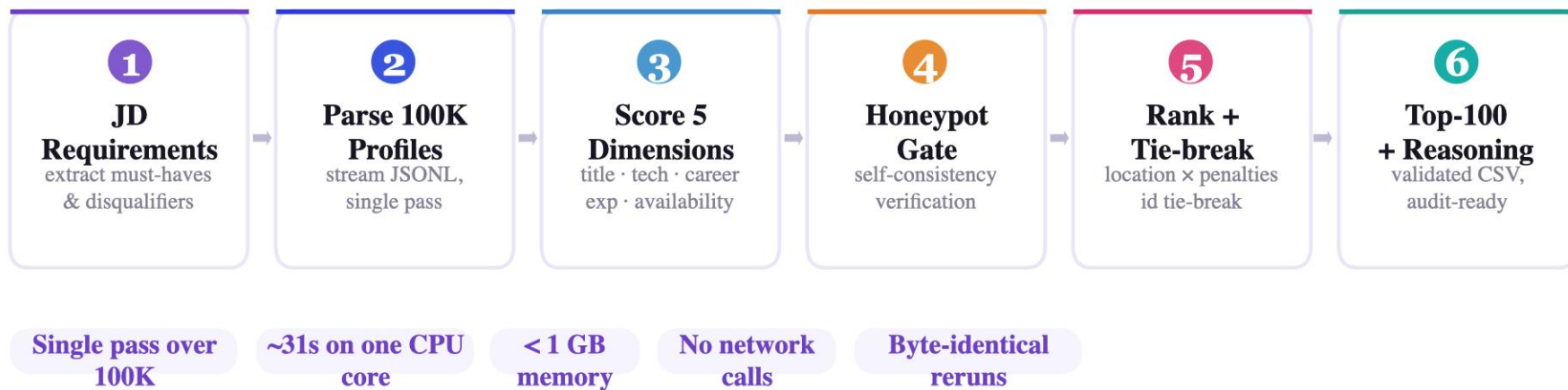
141 profiles flagged pool-wide

0 reached the submitted top-100 · DQ threshold is 10%

END-TO-END WORKFLOW

06

From JD input to ranked output



SYSTEM ARCHITECTURE

Four layers, one deterministic pass

07

INPUT

candidates.jsonl (100K profiles) + JD-derived rule configuration

**SCORING ENGINE**

five independent stdlib scoring modules

Title

Technical

Career

Experience

Availability

**VALIDATION**

skills cross-checked vs. career text · honeypot_flags() self-consistency

**AGGREGATION & OUTPUT**

weighted blend · location × penalties · honeypot gate · validated top-100 CSV + reasoning

GUARANTEES

- Deterministic
- Single pass
- Stdlib only
- < 1 GB RAM
- CPU only
- ~31 s

RESULTS & PERFORMANCE

08

Fast, clean, and within every budget

100,000**candidates scored**

complete pool, single pass

~31 s**end-to-end runtime**

one CPU core

0**external dependencies**

pure standard library

141**honeypots caught**

pool-wide consistency checks

0 %**honeypots in top-100**

DQ threshold is 10%

< 1 GB**peak memory**

16 GB budget, CPU-only

**CHALLENGE
CONSTRAINTS****Runtime**

31 s of 5 min

**CPU-only**

yes

**No network**

yes

**Deterministic**

verified

**Validator**

passes

TECHNOLOGIES USED

The simplest system you can trust

09

WHY STANDARD LIBRARY ONLY**Reproducibility**

No model weights, no version drift, no nondeterminism — every run is identical.

Portability

Runs anywhere Python 3.9+ runs: no GPU, no network, no install step.

Auditability

Judges can read the entire scoring logic line by line and verify it.

Inside the budget

Comfortably within the 5-minute, 16 GB, CPU-only envelope.

THE STACK**Python 3.9+**

json · csv · datetime · argparse · pathlib

Zero pip installs

requirements.txt is empty by design

No ML model

deterministic heuristics, not inference

python-pptx / Pillow

used only to build this deck

SUBMISSION ASSETS

Everything, reproducible in one command

10

■ GitHub repository[De-Coder05/redrob-ranking-challenge](#)

full source + this deck

■ Ranked output[InnoCoders.csv](#)

top-100, validator-clean

■ Approach deck[approach_deck.pdf](#)

this presentation

■ Submission metadata[submission_metadata.yaml](#)

mirrors the portal entry

■ Ranking engine[rank.py](#)

the entire system, stdlib-only

■ Format validator[validate_submission.py](#)

organizer-provided checks

REPRODUCE THE SUBMISSION

```
$ python rank.py --candidates ./candidates.jsonl --out ./InnoCoders.csv
```



INDIA.RUNS

Build what next India runs on

THANK YOU